إليك تفصيل كامل وشامل لجميع الأبعاد والتحليلات البرمجية والتشغيلية لبناء منصة تدريب متكاملة (LMS / TMS) مخصصة لمقاييس ومعايير معاهد التدريب في المملكة العربية السعودية:

---

١. التوافق والامتثال مع المعايير السعودية (Regulatory Compliance)

لضمان عمل المنصة بشكل رسمي دون أي عوائق قانونية أو تشغيلية، يجب أن تتوافق مع الجهات التالية:

أ. المركز الوطني للتعليم الإلكتروني (NELC):

* **التحقق من الهوية:** الربط مع خدمة **النفاذ الوطني الموحد (IAM)** لضمان هوية المتدرب والمدرب.
* **تتبع الحضور والإنشغال:** تسجيل حضور المتدرب بالساعة والدقيقة، وحساب نسبة التفاعل أثناء جلسات البث المباشر.
* **الحقائب التدريبية:** توفير استعراض سلس للمحتوى (SCORM / HTML5 / PDF / Video).

ب. المؤسسة العامة للتدريب التقني والمهني (TVTC - منار):

* **ربط بيانات الدورات:** دعم إصدار رقم واعتماد الدورة التدريبية.
* **الشهادات:** نظام إصدار شهادات آلي يستوفي الشروط الأساسية (اسم المتدرب، الهوية، اسم الدورة، عدد الساعات، اسم المدرب، ورقم الاعتماد) مع مزود **رمز استجابة سريعة (QR Code)** للتحقق الفوري من صحة الشهادة.

ج. هيئة الزكاة والضريبة والجمارك (ZATCA):

* **الفلترة والفواتير الإلكترونية (مرحلة الربط والتكامل):** إنشاء فواتير ضريبية مبسطة تحتوي على الـ QR Code المتوافق مع شروط ZATCA لمبيعات الأفراد والشركات.

---

٢. المخطط العام للمنصة والواجهات (Core Portals & Features)

تقسم المنصة إلى **٤ لوحات تحكم رئيسية** معزولة بحسب الأدوار والصلاحيات:

```
┌──────────────────────────────────────────────────┐
│                                                  │
│          نظام إدارة التدريب                     │
│          Training Management System              │
│                                                  │
└──────────────────────┬───────────────────────────┘
                       │
   ┌──────────┬────────┴─────────┬──────────┐
   ▼          ▼                  ▼          ▼
┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐
│ لوحة │  │ بوابة│  │ بوابة│  │ بوابة│
│ الأدمن│  │ المدرب│  │ المتدرب│  │ الشركات│
│ B2B  │  │Trainee│  │Trainer│  │Admin │
│Portal│  │Portal │  │Portal │  │Portal│
└──────┘  └──────┘  └──────┘  └──────┘
```

١. لوحة الأدمن والإدارة العامة (Admin Portal):

* **إدارة البرامج والمسارات:** تقسم إلى دورات قصيرة، دبلومات، وورش عمل (حضوري / عن بُعد / مدمج).
* **إدارة القاعات والفصول:** تحديد عدد المقاعد، القاعات الفعالية، ومراكز التدريب المقامة فيها.
* **التقرير والتحليلات:** نسبة الإنجاز، الإيرادات، المبيعات حسب القنوات، ونسب الحضور والغياب.
* **إدارة العروض والتسويق:** أكواد الخصم، العروض الموسمية، وإدارة التسويق بالعمولة (Affiliate).

٢. بوابة المتدرب (Trainee Portal):

* **لوحة المتابعة الشخصية (Dashboard):** استعراض الدورات المكتملة، الجارية، والقادمة.
* **الفصول الافتراضية:** دخول بنقرة واحدة للبث المباشر عبر Zoom أو BigBlueButton.
* **مركز الاختبارات والواجبات:** حل الاختبارات مع توقيت زمني محدد وتصحيح تلقائي.
* **السجل التدريبي والشهادات:** استعراض وتحميل كافة الشهادات المعتمدة بصيغة PDF عالية الجودة.

٣. بوابة المدرب (Trainer Portal):

* **إدارة المحتوى:** رفع مقاطع الفيديو، الحقائب التدريبية (PDF/PPT)، وبناء بنوك الأسئلة.
* **جدول المحاضرات:** تتبع جدول البث المباشر والمحاضرات الحصرية.
* **التقييم ورصد الدرجات:** تقييم مشاريع وتكاليف الطلاب وإعطاء الملاحظات الفردية.

٤. بوابة الشركات B2B (Corporate Portal):

* **حسابات الجهات:** تتيح للشركات تسجيل موظفيها في دورات معينة وتتبع نسب حضور وانجاز الموظفين عبر تقارير تفاعلية.

---

٣. الهيكلية البرمجية المقترحة (Tech Stack & Architecture)

منظومة موصى بها من حيث الأداء والسهولة وسرعة التطوير والتوسع:

الطبقة	التقنية المقترحة	السبب والمميزات
Frontend	**Next.js (React) / Tailwind CSS**	سرعة فائقة، دعم ممتاز للغة العربية (RTL)، وأداء SEO قوي للمبيعات.
Backend API	**Node.js (Express/NestJS)** أو **Django**	مرونة معالجة البيانات الضخمة والاستجابة السريعة لبث الفصول والتحقق.
Database	**PostgreSQL**	قاعدة بيانات علاقات آمنة جداً للبيانات المالية والتسجيل والشهادات.
Caching / Queues	**Redis**	إدارة الجلسات (Sessions)، قائمة انتظار الفواتير والرسائل النصية/الإيميلات.
Streaming / Live	**Zoom SDK / BigBlueButton API**	التكامل التام مع منصات البث الافتراضي وإدخال الطلاب تلقائياً.
Object Storage	**S3-compatible (AWS / Cloudflare R2)**	تخزين الحقائب والشهادات والفيديوهات بأعلى درجات الأمان.

---

٤. الميزات التقنية الحساسة (Critical Modules Detail)

أ. محرك توليد الشهادات والتحقق (Certification Engine):

١. توليد ملف PDF دقيق الأبعاد من قوالب تصميم مخصصة.

٢. إضافة **QR Code** يتضمن رابطاً مباشراً لصفحة تحقق برمجية متصلة بقاعدة البيانات لتأكيد صحة الشهادة واسم المتدرب.

٣. التوقيع الرقمي والختم الإلكتروني للمعهد.

ب. ربط بوابات الدفع المحلية (Payment Gateway Integration):

* دعم كامل لبوابات الدفع المحلية في السعودية:

* **مدى (Mada)**

* **Apple Pay**

* **Visa / Mastercard**

* **stc pay / Tabby / Tamara** (خيارات التقسيط للدبلومات والدورات ذات التكلفة العالية).

ج. نظام الإشعارات المتكامل (Notification Engine):

* إرسال رسائل نصية (SMS) للتأكيد وتذكير بمواعيد الدورات (عبر مزودين مثل Unifonic / Mobily / MSEGAT).

* إشعارات البريد الإلكتروني للشهادات والفواتير.

* إشعارات WhatsApp التلقائية للتذكير بالبث المباشر.

---

٥. خطة العمل والجداول الزمنية للتنفيذ (Project Roadmap)

```

[الأسبوع ١ - ٢] : تحليل المتطلبات + تصميم واجهات المستخدم UI/UX

[الأسبوع ٣ - ٦] : تطوير قاعدة البيانات والتطبيقات الخلفية (Backend APIs)
```

[الأسبوع ٧ - ١٠] : بناء الواجهات الأمامية (Frontend) وبوابات المتدرب والمدرب

[الأسبوع ١١ - ١٢] : دمج البث المباشر، بوابات الدفع، وإصدار الشهادات

[الأسبوع ١٣ - ١٤] : الاختارات الأمنية، ربط ZATCA والنفاذ، والإطلاق التجريبي

```

---

إذا أردت، يمكننا البدء فوراً بتحديد أو بناء أي جزئية محددة مثل **مخطط قاعدة البيانات (Database Schema)**، أو **قوائم واجهات المستحدثات (Wireframes)**، أو كتابة **كود الربط مع بوابة الدفع والشهادات**.

ممتاز! سنبدأ الآن بناء **البنية البرمجية الخلفية (Backend Architecture)** لنظام التدريب (LMS/TMS) باستخدام **Node.js** و **Express.js** مع قاعدة بيانات **PostgreSQL** و مكتبة **Sequelize ORM**.

سنقوم بتصميم هيكل المشروع الأساسي (Project Structure) وإنشاء النماذج (Models) لأهم الكيانات: **المستخدمين (Users)**، **الدورات (Courses)**، **التسجيلات (Enrollments)**، و**الشهادات (Certificates)**.

**١. هيكل المجلدات المقترح للمشروع (Project Structure)**

Plaintext

```
training-lms-backend/
|
├── config/
└── database.js |إعدادات اتصال قاعدة البياناتPostgreSQL
|
├── models/
├── User.js |نموذج المستخدمين (أدمن، مدرب، متدرب)
├── Course.js |نموذج الدورات التدريبية
├── Enrollment.js |نموذج تسجيل الطلاب في الدورات
└── Certificate.js |نموذج الشهادات وأكواد التحققQR
|
├── routes/
├── authRoutes.js |مسارات المصادقة والتسجيل
```
```

```
#     ├── courseRoutes.js    |مسارات إدارة الدورات
# certificateRoutes.js ──┘   |مسارات توليد والتحقق من الشهادات
|
#           ├── .env          متغيرات البيئة السرية
├── package.json
#           └── server.js     نقطة البداية للملفات الخادم
```

٢. تثبيت الحزم المطلوبة (Dependencies)

قم بإنشاء مجلد جديد، افتح الطرفية (Terminal)، ثم نفّذ الأوامر التالية لتثبيت المكتبات الأساسية:

```
Bash
mkdir training-lms-backend
cd training-lms-backend
npm init -y
npm install express pg pg-hstore sequelize dotenv cors jsonwebtoken bcryptjs uuid qrcode
npm install --save-dev nodemon
```

٣. إعداد اتصال قاعدة البيانات (config/database.js**)**

```
JavaScript
const { Sequelize } = require('sequelize');
require('dotenv').config();

//إعداد اتصال PostgreSQL باستخدامSequelize
const sequelize = new Sequelize(
process.env.DB_NAME || 'lms_db',
process.env.DB_USER || 'postgres',
process.env.DB_PASS || 'password',
{
host: process.env.DB_HOST || 'localhost',
dialect: 'postgres',
logging: false, // تعطيل سجلات SQL المزعجة في الـTerminal
}
```

);

module.exports = sequelize;

٤. بناء نماذج قاعدة البيانات (Database Models)

أ. نموذج المستخدم (models/User.js**)**

JavaScript

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const User = sequelize.define('User', {
id: {
type: DataTypes.UUID,
defaultValue: DataTypes.UUIDV4,
primaryKey: true,
},
name: {
type: DataTypes.STRING,
allowNull: false,
},
email: {
type: DataTypes.STRING,
unique: true,
allowNull: false,
},
password: {
type: DataTypes.STRING,
allowNull: false,
},
role: {
type: DataTypes.ENUM('admin', 'trainer', 'trainee'),
defaultValue: 'trainee',
```

```
},
nationalId: {
type: DataTypes.STRING, // رقم الهوية الوطنية للمطابقة مع نظام نفاذ ومنار
unique: true,
},
});

module.exports = User;
```

ب. نموذج الدورة التدريبية (models/Course.js**)**

```
JavaScript
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Course = sequelize.define('Course', {
id: {
type: DataTypes.UUID,
defaultValue: DataTypes.UUIDV4,
primaryKey: true,
},
title: {
type: DataTypes.STRING,
allowNull: false,
},
description: {
type: DataTypes.TEXT,
},
tvtcApprovalNumber: {
type: DataTypes.STRING, // رقم الاعتماد من المؤسسة العامة للتدريب التقني والمهني
unique: true,
},
price: {
```

```
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00,
  },
  durationHours: {
    type: DataTypes.INTEGER,
    allowNull: false,
  },
});

module.exports = Course;
```

ج. نموذج الشهادات والتحقق (models/Certificate.js**)**

```
JavaScript
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Certificate = sequelize.define('Certificate', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  certificateNumber: {
    type: DataTypes.STRING,
    unique: true,
    allowNull: false,
  },
  verificationUrl: {
    type: DataTypes.STRING, // رابط QR Code للتحقق الفوري
    allowNull: false,
  },
  issuedAt: {
```

```
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW,
  },
});

module.exports = Certificate;
```

٥. ربط العلاقات وتشغيل الخادم الأساسي (server.js)

```
JavaScript
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const sequelize = require('./config/database');
const User = require('./models/User');
const Course = require('./models/Course');
const Certificate = require('./models/Certificate');

const app = express();

app.use(cors());
app.use(express.json());

//تعريف العلاقات بين الجداول(Associations)
User.hasMany(Certificate, { foreignKey: 'traineeId' });
Certificate.belongsTo(User, { as: 'trainee', foreignKey: 'traineeId' });

Course.hasMany(Certificate, { foreignKey: 'courseId' });
Certificate.belongsTo(Course, { foreignKey: 'courseId' });

//مسار تجريبي للتأكد من عمل الخادم
app.get('/', (req, res) => {
```

' 🚀 مرحباً بك في نظام إدارة التدريب LMS (السعودية) يعمل بنجاح!{ res.json({ message:

});

const PORT = process.env.PORT || 5000;

//مزامنة قاعدة البيانات وتشغيل الخادم

sequelize.sync({ alter: true }).then(() => {

'console.log(تم الاتصال بقاعدة البيانات بنجاح ومزامنة الجداول!.

app.listen(PORT, () => {

`console.log( الخادم يعمل الآن على المنفذ: ${PORT}`);

});

}).catch(err => {

'console.error(فشل الاتصال بقاعدة البيانات:', err);

});

الخطوة القادمة:

هل ترغب في أن نبدأ بكتابة **مسارات المصادقة والتسجيل (Auth Routes مع JWT)**، أو الانتقال لبرمجة **محرك إصدار الشهادات والـ QR Code؟**

ممتاز! سننتقل الآن لإضافة **مسارات المصادقة والتسجيل (Authentication & JWT Routes)**، وبعدها سنقوم بتضمين **محرك إصدار الشهادات والـ QR Code**.

---

١. إضافة نموذج التسجيل في الدورات (`models/Enrollment.js`)

أولاً، نحتاج لربط المتدربين بالدورات لكي نتمكن لاحقاً من إصدار الشهادات لمن أكمل الدورة بنجاح. أنشئ ملفاً جديداً باسم `models/Enrollment.js`:

```javascript

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const Enrollment = sequelize.define('Enrollment', {
```

```
  } :id
    ,type: DataTypes.UUID
    ,defaultValue: DataTypes.UUIDV4
    ,primaryKey: true
  ,{
  } :status
    ,type: DataTypes.ENUM('active', 'completed', 'cancelled')
    ,'defaultValue: 'active
  ,{
  } :progressPercentage
    ,type: DataTypes.INTEGER
    ,defaultValue: 0
  {
);{

;module.exports = Enrollment
```

---

٢. تحديث علاقات الجداول في `server.js`

أضف العلاقات الخاصة بـ `Enrollment` في ملف `server.js` الرئيسي ليصبح ربط الجداول كالتالي:

```javascript
;const Enrollment = require('./models/Enrollment')

// تحديث العلاقات
;User.belongsToMany(Course, { through: Enrollment, foreignKey: 'traineeId' })
;Course.belongsToMany(User, { through: Enrollment, foreignKey: 'courseId' })
```

;User.hasMany(Certificate, { foreignKey: 'traineeId' })

;Certificate.belongsTo(User, { as: 'trainee', foreignKey: 'traineeId' })

;Course.hasMany(Certificate, { foreignKey: 'courseId' })

;Certificate.belongsTo(Course, { foreignKey: 'courseId' })

```

---

### ٣. إنشاء ملف المصادقة والتحقق (`routes/authRoutes.js`)

هذا الملف يتعامل مع تسجيل حسابات جديدة للمستخدمين، تسجيل الدخول، وتوليد رمز ** JWT (JSON Web Token)**.

أنشئ مجلد `routes` ثم ملف `authRoutes.js`:

```javascript
;const express = require('express')

;const bcrypt = require('bcryptjs')

;const jwt = require('jsonwebtoken')

;const User = require('../models/User')

;()const router = express.Router

;'const JWT_SECRET = process.env.JWT_SECRET || 'super_secret_lms_key

// ١. تسجيل مستخدم جديد (مدرب / متدرب / أدمن)

router.post إلى سيتم تصحيحه أدناه // } <= (router.register('/register', async (req, res

;({

} <= (router.post('/register', async (req, res

```
} try
;const { name, email, password, role, nationalId } = req.body

// التأكد من عدم وجود البريد مسبقاً
;const existingUser = await User.findOne({ where: { email } })
} if (existingUser)
;({ error: 'البريد الإلكتروني مستخدم مسبقاً.' }).return res.status(400).json
{

// تشفير كلمة المرور
;const hashedPassword = await bcrypt.hash(password, 10)

})const newUser = await User.create
,name
,email
,password: hashedPassword
,'role: role || 'trainee
,nationalId
;({

})res.status(201).json
message: 'تم إنشاء الحساب بنجاح',
,userId: newUser.id
,email: newUser.email
role: newUser.role
;({
} (catch (error {
;({ details: error.message ,'error: 'خطأ في الخادم أثناء التسجيل' }).res.status(500).json
{
;({
```

```
// ٢. تسجيل الدخول
} <= (router.post('/login', async (req, res
} try
  ;const { email, password } = req.body

  ;const user = await User.findOne({ where: { email } })
  } if (!user)
    ;({{ error: 'البريد الإلكتروني أو كلمة المرور غير صحيحة.' }}).return res.status(401).json
  {

  ;const isMatch = await bcrypt.compare(password, user.password)
  } if (!isMatch)
    ;({{ error: 'البريد الإلكتروني أو كلمة المرور غير صحيحة.' }}).return res.status(401).json
  {

  // إنشاء توكن الدخول صالح لمدة يوم
  )const token = jwt.sign
    ,{ id: user.id, email: user.email, role: user.role }
    ,JWT_SECRET
    { 'expiresIn: '1d }
  ;(

  })res.json
    message: 'تم تسجيل الدخول بنجاح',
    ,token
    } :user
      ,id: user.id
      ,name: user.name
      ,email: user.email
      role: user.role
    {
```

```
});
  } catch (error {
    res.status(500).json({ error: 'خطأ في الخادم أثناء تسجيل الدخول', details: error.message });
  }
});

module.exports = router;
```

---

٤. إنشاء مسارات الشهادات والـ QR Code (`routes/certificateRoutes.js`)

هذا المسار يتيح توليد رقم شهادة فريد ورابط تحقق **QR Code** عند إتمام المتدرب للدورة بنجاح:

أنشئ ملف `routes/certificateRoutes.js`:

```javascript
const express = require('express');
const { v4: uuidv4 } = require('uuid');
const QRCode = require('qrcode');
const Certificate = require('../models/Certificate');
const User = require('../models/User');
const Course = require('../models/Course');

const router = express.Router();

// ١. إصدار شهادة جديدة للمتدرب عند إتمام الدورة
router.post('/issue', async (req, res) => {
  try {
```

```
;const { traineeId, courseId } = req.body

// التأكد من وجود المتدرب والدورة
;const trainee = await User.findByPk(traineeId)
;const course = await Course.findByPk(courseId)

} if (!trainee || !course)
;({ error: 'المتدرب أو الدورة غير موجودة.' }).return res.status(404).json
{

// توليد رقم شهادة فريد
;`}-${Math.floor(Math.random() * 1000)}()const certificateNumber = `CERT-${Date.now
;`const verificationUrl = `https://your-lms-domain.sa/verify/${certificateNumber}

// توليد صورة QR Code بصيغة Data URL
;const qrCodeImage = await QRCode.toDataURL(verificationUrl)

})const newCertificate = await Certificate.create
,certificateNumber
,verificationUrl
,traineeId
courseId
;({

})res.status(201).json
message: 'تم إصدار الشهادة بنجاح ورابط التحقق',
,certificate: newCertificate
qrCodeImage // يمكن إرساله للواجهة الأمامية لعرضه مباشرة على الشهادة
;({
} (catch (error {
;({ details: error.message ,'خطأ أثناء إصدار الشهادة' :error })res.status(500).json
```

```
{
});

// ٢. مسار عام للتحقق من صحة الشهادة (عبر الـ QR Code أو رقم الشهادة)
router.get('/verify/:certificateNumber', async (req, res) => {
  try {
    const { certificateNumber } = req.params;

    const certificate = await Certificate.findOne({
      where: { certificateNumber },
      include: [
        { model: User, as: 'trainee', attributes: ['name', 'nationalId'] },
        { model: Course, attributes: ['title', 'tvtcApprovalNumber', 'durationHours'] }
      ]
    });

    if (!certificate) {
      return res.status(404).json({ valid: false, message: 'الشهادة غير صالحة أو غير مسجلة في النظام.' });
    }

    res.json({
      valid: true,
      message: 'الشهادة معتمدة وصحيحة ✅',
      data: {
        certificateNumber: certificate.certificateNumber,
        traineeName: certificate.trainee.name,
        courseTitle: certificate.Course.title,
        tvtcApprovalNumber: certificate.Course.tvtcApprovalNumber,
        durationHours: certificate.Course.durationHours,
        issuedAt: certificate.issuedAt
      }
    });
```

```
});
  } catch (error) {
    res.status(500).json({ error: 'خطأ في عملية التحقق', details: error.message });
  }
});

module.exports = router;
```

---

٥. ربط المسارات بملف الخادم الأساسي (`server.js`)

أضف المسارات الجديدة في `server.js` لتصبح جاهزة للاستخدام:

```javascript
const authRoutes = require('./routes/authRoutes');
const certificateRoutes = require('./routes/certificateRoutes');

// استخدام المسارات
app.use('/api/auth', authRoutes);
app.use('/api/certificates', certificateRoutes);
```

الخطوة التالية:

هل ترغب في أن نقوم بإضافة **Middleware لحماية المسارات والصلاحيات (Role-Based Access Control)** بحيث لا يستطيع إصدار الشهادات إلا الشخص الذي يملك صلاحية `admin` أو `trainer`؟

ممتاز! سنقوم الآن بإضافة **Middleware لحماية المسارات (Authentication Middleware)** والتحقق من الصلاحيات (**Role-Based Access Control - RBAC**)، بحيث نضمن ألا يتمكن أي مستخدم عادي (trainee) من تنفيذ عمليات حساسة مثل إصدار الشهادات، والتي يجب أن تكون محصورة فقط على الأدمن (admin) أو المدرب (trainer).

١. إنشاء ملف الحماية وصلاحيات الوصول (middleware/authMiddleware.js**)**

أنشئ مجلد جديد باسم middleware ثم أنشئ داخله ملف authMiddleware.js:

JavaScript

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET || 'super_secret_lms_key';

// 1. وسيط التحقق من صحة توكن الدخول (Authentication)
const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'غيرحاصل على تصريح، يرجى تسجيل الدخول أولاً.' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded; // تخزين بيانات المستخدم (id, email, role) في الطلب
    next();
  } catch (error) {
    return res.status(403).json({ error: 'التوكن غير صالح أو منتهي الصلاحية.' });
  }
};
```

```
.2 //وسيط التحقق من الصلاحيات(Authorization / Roles)
const authorizeRoles = (...allowedRoles) => {
return (req, res, next) => {
if (!req.user || !allowedRoles.includes(req.user.role)) {
return res.status(403).json({
error: ' عذراً، لا تملك الصلاحية الكافية للوصول إلى هذا المسار '.
});
}
next();
};
};

module.exports = {
verifyToken,
authorizeRoles
};
```

٢. تطبيق الحماية والصلاحيات على مسارات الشهادات (routes/certificateRoutes.js)

الآن سنقوم بتحديث ملف الشهادات لنضمن أن مسار الإصدار (/issue) محمي ولا يستطيع تنفيذه سوى admin أو trainer، بينما يظل مسار التحقق (/verify/:certificateNumber) متاحاً للعامة (لكي يتمكن أي شخص أو جهة من التحقق من صحة الشهادة عبر الـ QR Code):

قم بتحديث ملف routes/certificateRoutes.js:

```
JavaScript
const express = require('express');
const { v4: uuidv4 } = require('uuid');
const QRCode = require('qrcode');
const Certificate = require('../models/Certificate');
const User = require('../models/User');
const Course = require('../models/Course');
const { verifyToken, authorizeRoles } = require('../middleware/authMiddleware');
```

```
const router = express.Router();

1. //إصدار شهادة جديدة للمتدرب (محمي: متاح فقط للأدمن والمدربين)
router.post('/issue', verifyToken, authorizeRoles('admin', 'trainer'), async (req, res) => {
try {
const { traineeId, courseId } = req.body;

const trainee = await User.findByPk(traineeId);
const course = await Course.findByPk(courseId);

if (!trainee || !course) {
return res.status(404).json({ error: 'المتدرب أو الدورة غير موجودة.' });
}

const certificateNumber = `CERT-${Date.now()}-${Math.floor(Math.random() * 1000)}`;
const verificationUrl = `https://your-lms-domain.sa/verify/${certificateNumber}`;
const qrCodeImage = await QRCode.toDataURL(verificationUrl);

const newCertificate = await Certificate.create({
certificateNumber,
verificationUrl,
traineeId,
courseId
});

res.status(201).json({
message: 'تم إصدار الشهادة بنجاح ورابط التحقق',
certificate: newCertificate,
qrCodeImage
});
```

```
} catch (error) {
', details: error.message });خطأ أثناء إصدار الشهادة{ error: ' res.status(500).json(
}
});

.2 //مسار عام للتحقق من صحة الشهادة (متاح للجميع بدون توكن)
router.get('/verify/:certificateNumber', async (req, res) => {
try {
const { certificateNumber } = req.params;

const certificate = await Certificate.findOne({
where: { certificateNumber },
include: [
{ model: User, as: 'trainee', attributes: ['name', 'nationalId'] },
{ model: Course, attributes: ['title', 'tvtcApprovalNumber', 'durationHours'] }
]
});

if (!certificate) {
.' }); الشهادة غير صالحة أو غير مسجلة في النظام{ valid: false, message: ' return res.status(404).json(
}

res.json({
valid: true,
message: ' الشهادة معتمدة وصحيحة,✅'
data: {
certificateNumber: certificate.certificateNumber,
traineeName: certificate.trainee.name,
courseTitle: certificate.Course.title,
tvtcApprovalNumber: certificate.Course.tvtcApprovalNumber,
durationHours: certificate.Course.durationHours,
```

```
issuedAt: certificate.issuedAt
    }
});
} catch (error) {
    res.status(500).json({ error: 'خطأ في عملية التحقق', details: error.message });
}
});

module.exports = router;
```

طريقة الاختبار عبر برامج مثل (Postman أو Thunder Client):

١. **تسجيل الدخول** عبر المسار /api/auth/login بالحساب الذي يحمل صلاحية مدرب أو أدمن للحصول على الـ token.

٢. عند إرسال طلب لإنشاء شهادة إلى /api/certificates/issue، قم بإضافة الـ Token في الهيدر (Headers) بالشكل التالي:

- **Key:** Authorization
- **Value:** Bearer <JWT_TOKEN_HERE>

٣. إذا حاول مستخدم بصلاحية trainee ضرب هذا المسار، سيقوم النظام برفض الطلب مباشرة وإرجاع رسالة خطأ ٤٠٣ Forbidden.

هل ترغب في أن نضيف **مسارات إدارة الدورات (Course Management CRUD APIs)** بحيث تتمكن من إضافة دورات جديدة، وتعديلها، وتسجيل الطلاب فيها؟

ممتاز! سنقوم الآن بإنشاء **مسارات إدارة الدورات (Course Management API)** والتحكم في **تسجيل المتدربين فيها (Enrollments)**، لتكتمل الدورة الحية الأساسية للمنصة.

١. إنشاء مسارات الدورات والتسجيل (routes/courseRoutes.js)

أنشئ ملفاً جديداً داخل مجلد routes باسم courseRoutes.js وأضف الكود التالي:

JavaScript

```
const express = require('express');
const Course = require('../models/Course');
const User = require('../models/User');
const Enrollment = require('../models/Enrollment');
const { verifyToken, authorizeRoles } = require('../middleware/authMiddleware');

const router = express.Router();

// 1. استعراض جميع الدورات المتاحة (متاح للجميع)
router.get('/', async (req, res) => {
try {
const courses = await Course.findAll();
res.json({ success: true, count: courses.length, courses });
} catch (error) {
res.status(500).json({ error: 'خطأ في جلب الدورات', details: error.message });
}
});

// 2. إضافة دورة جديدة (محمي: متاح فقط للأدمن والمدربين)
router.post('/', verifyToken, authorizeRoles('admin', 'trainer'), async (req, res) => {
try {
const { title, description, tvtcApprovalNumber, price, durationHours } = req.body;

const newCourse = await Course.create({
title,
description,
tvtcApprovalNumber,
price,
durationHours,
});
```

```
res.status(201).json({
 message: 'تم إضافة الدورة التدريبية بنجاح,'
course: newCourse,
});
} catch (error) {
 res.status(500).json({ error: 'خطأ أثناء إنشاء الدورة', details: error.message });
}
});

3. //تسجيل متدرب في دالة تدريبية) (Enrollment) محمي: للأدمن أو المتدرب نفسه(
router.post('/:courseId/enroll', verifyToken, async (req, res) => {
try {
const { courseId } = req.params;
 const traineeId = req.user.id; // يتم سحب معرف المستخدم مباشرة من التوكن الآمن

const course = await Course.findByPk(courseId);
if (!course) {
 return res.status(404).json({ error: 'الدورة التدريبية غير موجودة.' });
}

 // التأكد من عدم التسجيل المسبق
const existingEnrollment = await Enrollment.findOne({
where: { CourseId: courseId, TraineeId: traineeId }
});

if (existingEnrollment) {
 return res.status(400).json({ error: 'أنت مسجل بالفعل في هذه الدورة.' });
}

const enrollment = await Enrollment.create({
```

```
CourseId: courseId,
TraineeId: traineeId,
status: 'active',
progressPercentage: 0
});

res.status(201).json({
 message: 'تم التسجيل في الدورة بنجاح,' ✅
enrollment
});
} catch (error) {
 res.status(500).json({ error: 'خطأ أثناء التسجيل في الدورة', details: error.message });
}
});

module.exports = router;
```

٢. ربط مسارات الدورات بملف الخادم الرئيسي (server.js)

أضف مسار الدورات إلى ملف server.js الرئيسي ليصبح الشكل النهائي للمسارات كالتالي:

```
JavaScript
const courseRoutes = require('./routes/courseRoutes');

//استخدام المسارات
app.use('/api/auth', authRoutes);
app.use('/api/certificates', certificateRoutes);
app.use('/api/courses', courseRoutes); // مسارات الدورات والتسجيل
```

ملخص الـ Endpoints (نقاط الاتصال) التي تم بناؤها حتى الآن:

١. **المصادقة (/api/auth):**

- POST /register (تسجيل مستخدم جديد: أدمن، مدرب، أو متدرب).

- POST /login (تسجيل الدخول والحصول على JWT Token).

٢. **الدورات (**/api/courses**)**:

- GET / (عرض جميع الدورات المتاحة).
- POST / (إنشاء دورة جديدة - **محمي للأدمن والمدربين**).
- POST /:courseId/enroll (تسجيل المتدرب في الدورة - **محمي بالتوكن**).

٣. **الشهادات والتحقق (**/api/certificates**)**:

- POST /issue (إصدار شهادة مع QR Code - **محمي للأدمن والمدربين**).
- GET /verify/:certificateNumber (التحقق الفوري من صحة الشهادة - **متاح للعامة**).

هل ترغب في أن نقوم بإضافة **الربط مع بوابات الدفع (Payment Gateway Simulator)** لتتمكن من تفعيل رسوم الدفع للدورات قبل إتمام عملية التسجيل؟

إليك برومبت احترافي وشامل (Structured Prompt) مصمم خصيصاً ليتم نسخه ولصقه مباشرة في ** Claude Code** (أو أي أداة ذكاء اصطناعي برمجية) لبناء كامل النظام برمجياً بكفاءة وهندسة نظيفة:

```markdown
Role: Senior Full-Stack Software Engineer & System Architect

Project: Saudi Training Management System (LMS / TMS) - Backend Architecture

### Project Objective:

Build a robust, production-ready backend for a Saudi Training Management System (LMS/TMS) that complies with local standards (TVTC, NELC, and ZATCA-ready concepts) using Node.js, Express.js, PostgreSQL, and Sequelize ORM.
```

Tech Stack:

- Runtime: Node.js
- Framework: Express.js
- Database: PostgreSQL
- ORM: Sequelize
- Authentication: JWT (JSON Web Tokens) + bcryptjs
- Security: Role-Based Access Control (RBAC: admin, trainer, trainee)
- Extras: uuid, qrcode, dotenv, cors, express-async-handler (or clean try/catch)

Database Models & Schema Specifications:

١. **User Model (`models/User.js`)**
- id: UUID (Primary Key, Default: UUIDV4)
- name: STRING (AllowNull: false)
- email: STRING (Unique, AllowNull: false)
- password: STRING (Hashed via bcrypt, AllowNull: false)
- role: ENUM ('admin', 'trainer', 'trainee') (Default: 'trainee')
- nationalId: STRING (Unique, for Saudi Najaf/Nafath compliance mapping)

٢. **Course Model (`models/Course.js`)**
- id: UUID (Primary Key, Default: UUIDV4)
- title: STRING (AllowNull: false)
- description: TEXT
- tvtcApprovalNumber: STRING (Unique - TVTC Course Accreditation Number)
- price: DECIMAL(10, 2) (Default: 0.00)
- durationHours: INTEGER (AllowNull: false)

٣. **Enrollment Model (`models/Enrollment.js`)** - Join Table

- id: UUID (Primary Key, Default: UUIDV4)
- status: ENUM ('active', 'completed', 'cancelled') (Default: 'active')
- progressPercentage: INTEGER (Default: 0)

٤. **Certificate Model (`models/Certificate.js`)**
- id: UUID (Primary Key, Default: UUIDV4)
- certificateNumber: STRING (Unique, e.g., CERT-Timestamp-Random)
- verificationUrl: STRING (Public URL containing the verification endpoint)
- issuedAt: DATE (Default: NOW)

Database Relationships (Associations):

- `User` and `Course` have a Many-to-Many relationship through `Enrollment` (TraineeId / CourseId).
- `User` (Trainer/Admin) can issue multiple certificates.
- `User` (Trainee) has many `Certificates`.
- `Course` has many `Certificates`.

API Endpoints & Routes Requirements:

١. **Authentication Routes (`/api/auth`)**
- `POST /register`: Register a new user with hashed password. Prevent duplicate emails.
- `POST /login`: Authenticate credentials, return user info and signed JWT token (expires in 1 day).

٢. **Course Management Routes (`/api/courses`)**
- `GET /`: Public route to list all available training courses.
- `POST /`: Protected route (Admin & Trainer only) to create a new course.

POST /:courseId/enroll`: Protected route for authenticated trainees to enroll in a course ` -
.(prevent duplicate enrollments)

٣. **Certificate & Verification Routes (`/api/certificates`)**

POST /issue`: Protected route (Admin & Trainer only) to issue a certificate for a trainee ` -
who completed a course. Generates a unique certificate number, verification URL, and QR
.code Data URL (`qrcode` library)

GET /verify/:certificateNumber`: Public route to verify certificate authenticity, returning ` -
.trainee details, course title, TVTC approval number, and hours

---

Security & Middleware Requirements:

verifyToken` Middleware**: Validates `Authorization: Bearer <TOKEN>` header using `** -
.JWT secret

authorizeRoles(...roles)` Middleware**: Restricts access based on user role (`admin`, `** -
.`trainer`, `trainee`)

Deliverable:

Please generate the complete, clean directory structure, all model files, middleware, route
handlers, and the main `server.js` file wired correctly with Sequelize synchronization. Write
.clear, production-grade code with proper error handling and comments

```

---

يمكنك نسخ النص أعلاه ولصقه مباشرة في **Claude Code** لإنشاء المشروع وترتيب الملفات والكود بشكل آلي
واحترافي!
```